Coach First

Change Record — Claude Code session

3 August 2026 · branch **codex/full-product-readiness** · handoff for Codex

This document records every change made to the repository in this session, the verification performed, and the state of the production database as observed. It is written so that Codex can resume without re-deriving context.

> **Summary**
>
> Three commits on branch **fix/analyst-access-control**. **1b06607** closes a fail-open authorisation hole that admitted any account with no organization membership into the full internal analyst application, and removes the legacy personal-club setup gate. **e1dbaed** makes the corpus shared across the Coach First team at the row-level-security layer. **bb510da** removes the 243 application filters that were re-narrowing it back to the row's creator. Nothing has been pushed and no production data or schema was modified.

1. Starting state

The branch was already merged: **origin/main** and **HEAD** were identical. Only the local **main** was stale, which made the branch look like 15 unmerged commits. No merge work was required or performed.

Baseline health was verified before any edit was made, and all gates were green:

Gate	Result
tsc --noEmit	clean
eslint .	clean
npm test	74 / 74 passing
npm run build	succeeds
npm run verify:production	passes against the live deployment

2. The defect that was fixed

Fail-open analyst authorisation

src/lib/supabase/middleware.ts guarded the internal analyst application with two checks only: **isClubOnlyIdentity** and **isCoachOnlyIdentity**. Both are derived in **classifyOrganizationAccess** and both are **false** when an account holds no organization membership at all.

An authenticated account with no membership therefore satisfied neither guard and passed straight through into every analyst route and every analyst API route. Access was granted by the *absence* of an external identity rather than by the presence of an internal one.

> **Why this mattered in production, not just in theory**
>
> Three real accounts in the production database hold no membership: **ryanaston1@gmail.com**, **coachapp.demo2026@gmail.com** and **codex.apifootball…@gmail.com**. Each of them could reach the internal analyst application. Row-level security still limited them to rows they owned, so the trusted corpus was not readable by them — but they received the entire internal UI and could create clubs, coaches and mandates.

Legacy personal-club setup gate

src/app/(dashboard)/layout.tsx required every dashboard user to own a row in **clubs** with **user_id = auth.uid()**, redirecting to **/dashboard/setup** otherwise. This predates the organization model. Its effect was that

a correctly invited internal analyst was still forced to invent a personal club before seeing the product, and landed in an empty workspace afterwards. This is the redirect and blank-setup symptom recorded in July.

3. Changes made

File	Change
src/lib/organizations/access.ts modified	Added **hasNoWorkspaceIdentity** to **OrganizationAccessProfile**. Added **canEnterAnalystApplication()** as the single authority for analyst entry (returns **hasActiveInternalAccess**). Added **resolveWorkspaceHome()** to map an account to its correct home, and the **NO_WORKSPACE_PATH** constant.
src/lib/supabase/middleware.ts modified	Analyst routes and analyst API routes now require **canEnterAnalystApplication()**. Added a guard sending accounts with no workspace to **/no-access**, and a reverse guard keeping accounts that do have a workspace off that page. The **/login** destination now goes through **resolveWorkspaceHome()** instead of an inline ternary.
src/app/(dashboard)/layout.tsx modified	Removed the mandatory personal-club gate and its **/dashboard/setup** redirects entirely. Added a defence-in-depth **canEnterAnalystApplication()** check behind the middleware. The **headers()** import and pathname read are no longer needed and were dropped.
src/app/no-access/page.tsx new	Destination for an authenticated account with no workspace. Explains that access is issued by invitation. Statically prerendered.
src/app/no-access/_components/no-access-sign-out.tsx new	Client sign-out button, following the existing **inactive-club-sign-out.tsx** pattern.
src/lib/organizations/access.test.ts modified	Four tests added covering the closed hole. Two existing **deepEqual** assertions updated for the new profile field.
supabase/migrations/20260803120000_internal_team_corpus_access.sql new	Adds **is_internal_corpus_operator()** and one shared-corpus policy across 45 tables. Carries the excluded-table list and the reasoning for each exclusion in its header comment.
supabase/tests/internal_corpus_rls.sql new	Contract test for the above. Asserts a second analyst reads and writes shared corpus, that a club identity and a membership-less account are denied, and that **coach_private_materials** never acquires the policy.
package.json modified	Adds **test:rls:corpus** and includes it in **test:rls**, so CI runs it wherever **SUPABASE_DB_PASSWORD** is configured.
89 application files modified	243 corpus **eq('user_id', …)** filters removed; helpers renamed from **…ForUser** to **…ForTeam**; two **integration_sync_state** readers made deterministic. See section 8.

> **Deliberate design decision to preserve**
>
> Analyst entry is now granted by **active internal membership only**. It is never granted by the absence of another identity. If Codex adds a new identity type, the correct change is to extend **resolveWorkspaceHome()**, not to add another negative check in the middleware.

4. Failure-mode change — verified, not assumed

This fix inverts a failure mode and that inversion was checked directly against production before being accepted.

Previously, if the **organization_memberships** query returned nothing for any reason, every derived flag was false and the user was **admitted**. Now the same condition **denies**. Had row-level security prevented a user from reading their own memberships, this change would have locked out every account.

The select policy on **organization_memberships** is:

```
(user_id = (SELECT auth.uid()))
  OR is_organization_member(organization_id, ARRAY['owner','admin',
                            'club_owner','club_director'])
  OR is_internal_operator()
```

The leading term guarantees a self-read. This was then confirmed empirically by executing queries under **role authenticated** with impersonated JWT claims inside a rolled-back transaction:

Account	Memberships visible to itself	Resulting access
jakemarques@live.com	**owner** (active), **club_owner** (active)	Analyst application retained
coachapp.demo2026@gmail.com	0 rows	Correctly routed to **/no-access**

There is therefore no lockout risk for the owner account.

5. Verification performed

Check	Result
npm test	78 / 78 passing (74 before, 4 added)
tsc --noEmit	clean
eslint .	clean
npm run build	succeeds; **/no-access** registered as a static route
Unauthenticated **/dashboard**	307 → **/login**
Unauthenticated **/mandates**	307 → **/login**
Unauthenticated **/no-access**	307 → **/login**
Unauthenticated analyst API	307 → **/login**
/no-access prerendered output	Renders heading, explanatory copy and sign-out control
RLS self-read	Confirmed under impersonated JWT, see section 4

Not verified: the authenticated no-workspace journey was not exercised end to end in a browser, because doing so requires signing in as one of those accounts and credentials were not used in this session. The routing decision is covered by unit tests and the page output was verified from the prerender.

6. Production database state as observed

Project **Coach Matchmaker** (**mkqpvugcohmvhgwdiwat**), read only. Two organizations exist: **Coach First** (internal) and **West Ham United** (club).

Account	Coaches	Clubs	Mandates	Memberships
jakemarques@live.com	194	95	1	Coach First: owner West Ham United: club_owner
codex.apifootball…@gmail.com	3	2	0	**none**
ryanaston1@gmail.com	0	0	0	**none**
coachapp.demo2026@gmail.com	0	0	0	**none**

7. Action required before this ships

The three membership-less accounts above will now reach **/no-access** instead of the analyst application. That is the intended behaviour, but it means **the partner and demo accounts must be granted membership or they will see nothing at all**.

There is currently **no in-app path to grant internal membership**. Club and coach invitations exist, but adding a Coach First analyst is a direct database operation. Suggested statement, to be reviewed before running:

```
insert into public.organization_memberships
  (organization_id, user_id, role, status)
select o.id, u.id, 'analyst', 'active'
from public.organizations o, auth.users u
where o.slug = 'coach-first'
  and u.email = 'ryanaston1@gmail.com'
on conflict do nothing;
```

Decide separately what the demo account should be: an internal analyst, or a club identity attached to West Ham United. The two give very different product experiences.

> **Also outstanding**
>
> The **codex.apifootball** account owns 3 coaches and 2 clubs of orphaned test data inside the production corpus. It was left untouched. It should either be reassigned to Coach First or deleted — but that is a destructive production data change and was not made unilaterally.

8. Team-scoping the corpus — addressed

The problem

Every core table — **coaches**, **clubs**, **mandates** and the intelligence tables — is still owned by an individual user through **user_id**, enforced by the February 2026 policies in **20260227_ownership_bootstrap.sql**:

```
create policy "Users can view own coaches" on public.coaches
  for select using (auth.uid() = user_id);
```

No later migration widened this to the internal team. There are **281 eq('user_id', …)** filters across roughly 90 files reinforcing it at the application layer.

Consequence: two Coach First analysts cannot see the same coaches, clubs or mandates. The corpus that **REAL_PRODUCT_WORKFLOW.md** calls “the moat” is private to whoever typed it in. The production data already shows the split — 197 coaches and 97 clubs fragmented across two owners.

How scope was decided

The **user_id** filters are **two different concerns wearing the same name**. Separating them was the actual work, and it was done per table rather than by pattern match:

Kind	Example tables	Correct treatment
Corpus scope — should become team-wide	coaches, clubs, mandates, intelligence items	Widen to the internal organization
Identity scope — must stay per user	organization_memberships, external_identity_profiles, coach portal records	**Leave exactly as is**

A blind sweep would widen the second category and create a genuine data-leak vulnerability across external identities.

What was built

Migration 20260803120000_internal_team_corpus_access.sql. is_internal_operator() already keys on **organization_type = 'internal'** rather than a specific organization id, and exactly one internal organization exists, so this is a policy change rather than a multi-tenant migration — no **organization_id** column was added to **clubs** or **coaches**. A new **is_internal_corpus_operator()** helper admits owner, admin and analyst, and a single policy is applied across 45 corpus tables. Because permissive policies OR together, the existing per-user policies are untouched and still serve non-internal callers.

Application sweep. 243 **eq('user_id', …)** filters removed across 89 files, restricted to an explicit corpus allowlist. Helpers whose meaning changed were renamed: **getClubsForUser** and its siblings returned the caller's rows and now return the team's, so they became **getClubsForTeam** and take no user argument. **user_id** is still written on insert as created-by.

Two hazards were checked for explicitly before applying:

Hazard	Finding
An **update** or **delete** whose only predicate was **user_id** would become an all-rows write	**Zero instances.** Every write chain also carries **.eq('id', …)**
A **.single()** that was unique only because of the user filter would start throwing	36 candidates, all but two are inserts returning one row by construction. The two real ones read **integration_sync_state**, whose uniqueness is **(user_id, sync_key)**; they now take the most recent row

Deployment order does not matter. With the migration applied but the code not yet deployed, the application still filters and behaviour is unchanged. With the code deployed but the migration not applied, row level security is still per-user and behaviour is unchanged. Sharing begins only once both are live.

Verification

Check	Result
Second internal analyst	Reads **and writes** corpus created by another internal member
Club-only identity	0 rows across clubs, coaches, mandates, intelligence
Membership-less account	Still only its own 3 coaches, not the team's 197
coach_private_materials	Asserted never to carry the shared-corpus policy
tsc / **eslint** / **npm test** / **build**	clean / clean (0 warnings) / 78 passing / succeeds

The database assertions ran against production inside transactions that were rolled back, and are captured permanently in **supabase/tests/internal_corpus_rls.sql**, wired into **npm run test:rls**.

> **Consequence to be aware of**
>
> Any internal analyst can now edit or delete any team record. There is no per-record ownership check left on the corpus. That is the correct model for shared working material and matches **REAL_PRODUCT_WORKFLOW.md**, but if analysts should instead be read-only on records they did not create, that is a further policy change and has not been made.

Still outstanding: the orphaned **codex.apifootball** rows (3 coaches, 2 clubs) are owned by an account with no membership, so they remain invisible to the team and are now effectively stranded. They should be reassigned to Coach First or deleted.

> **Migrations auto-deploy**
>
> **.github/workflows/supabase-migrations.yml** runs **supabase db push** against production on every push to **main**, after lint, typecheck, test and build pass. Any migration written for the work above reaches the live database as soon as it is merged. Treat that workflow as the deployment gate.

9. Smaller observations, not acted on

Item	Detail
/dashboard/setup is now unreferenced	Nothing links to it after the gate removal. The page remains reachable by direct URL and still performs a client-side insert into **clubs**. It is inconsistent with the organization model and is a deletion candidate.
Demo seeding was removed	Commit **b7da3c8** deleted **seed-demo-impl.ts**, **demo-narrative.ts** and the generate button, roughly 1,600 lines. **clear-my-data.ts** survived. There is now no way to populate a fresh account for a demonstration.
Test shape	13 test files against 347 source files, all pure-logic contract tests. Nothing exercises a rendered page or a server action end to end.
Untracked files	Roughly 21 MB across **outputs/**, **tmp/** and **.claire/** is untracked and not in **.gitignore**.

10. Repository state at handoff

Three commits on branch **fix/analyst-access-control**, branched from **codex/full-product-readiness**. **Nothing has been pushed.**

```
bb510da  Read the corpus as a team rather than per user
e1dbaed  Share the internal corpus across the Coach First team
1b06607  Require internal membership to enter the analyst application
```

No production data or schema was modified. Every database statement was either read-only or executed inside a transaction that was rolled back, including the migration rehearsals and the impersonated-role assertions.

Two things must happen before the corpus change has any effect: the migration has to reach the database, and the membership rows in section 7 have to be created. Until an account holds an active internal membership it sees the no-workspace page, and until the migration runs the corpus stays per-user.

> **Suggested first step on picking this up**
>
> Run **npm run test:rls** against a database with the migration applied. It exercises the whole contract — second-analyst read and write, club identity denied, membership-less account denied, and coach private material left behind its release boundary.